

Chuẩn bị môi trường học

Làm **một lần** trước buổi 1. Mất khoảng 20–30 phút (phần lớn là chờ tải). Mỗi buổi học sau đó chỉ cần `make up`.

Hướng dẫn kiểm theo tài liệu chính thức ngày **2026-09-17**. Lệnh cài có thể đổi — nếu một lệnh không chạy, mở link nguồn ghi cạnh lệnh đó.

Cần gì

Thành phần	Bắt buộc?	Dùng cho
Linux, macOS, hoặc Windows + WSL2	có	cả khoá — lệnh trong khoá là lệnh Linux/macOS
uv	có	tự cài Python 3.12 và thư viện của từng buổi
make, git, curl	có	<code>make up / check / down</code>
CPU 4 nhân, 16 GB RAM, 30 GB đĩa trống	khuyến nghị	8 GB RAM đủ cho buổi 1–28 và 38–43
API key OpenAQ, EIA (miễn phí)	buổi 10, 41–44, dự án	dữ liệu không khí, điện
Tài khoản Kaggle	buổi 23, đề A	dữ liệu M5 (có bộ dự phòng mở)
Ollama hoặc llama.cpp	buổi 36–37	chạy LLM mở trên máy, không tốn tiền API
Docker	buổi 41–43	đóng gói pipeline và API

Không cần cài Python, Jupyter hay thư viện nào bằng tay: uv làm hết, riêng cho từng buổi.

1. Windows: bật WSL2 trước

Cần Windows 10 bản 2004 trở lên (build 19041+) hoặc Windows 11. Mở **PowerShell quyền Administrator**:

```
wsld --install
```

Khởi động lại máy. Lệnh này bật WSL và cài bản phân phối Ubuntu mặc định. Từ đây mọi lệnh trong khoá chạy **trong cửa sổ Ubuntu**, không chạy trong PowerShell.

Hai điều nên làm ngay:

- **Để thư mục khoá học trong hệ thống tệp Linux** (`/home/<tên>/...`), không để ở `/mnt/c/...`. Microsoft khuyến nghị: "store your files in the WSL file system if you are working in a Linux command line" — đọc ghi tệp nhanh hơn nhiều.
- **Giới hạn RAM cho WSL** nếu máy yếu. Mặc định WSL được dùng 50% RAM của Windows. Tạo tệp `%UserProfile%\wslconfig`:

```
[wsl2]
memory=12GB
```

rồi chạy `wsl --shutdown` trong PowerShell để áp dụng.

Nguồn: learn.microsoft.com/windows/wsl/install, [.../wsl/filesystems](https://learn.microsoft.com/windows/wsl/filesystems), [.../wsl/wsl-config](https://learn.microsoft.com/windows/wsl/wsl-config).

2. Công cụ hệ thống: make, git, curl

Ubuntu / WSL:

```
sudo apt update
sudo apt install -y build-essential git curl unzip
```

macOS: `xcode-select --install` (cài Command Line Tools: git, make). Bản make đi kèm macOS là GNU Make 3.81 — đủ cho Makefile của khoá, vì mọi Makefile chỉ dùng cú pháp 3.81.

Kiểm tra:

```
make --version | head -1
git --version
```

3. uv — quản lý Python và thư viện

```
curl -LsSf https://astral.sh/uv/install.sh | sh # Linux, macOS, WSL
# hoặc trên macOS: brew install uv
```

Mở terminal mới rồi kiểm tra:

```
uv --version # khoá kiểm với uv 0.12.15
uv python install 3.12 # tải sẵn Python 3.12 (không bắt buộc – make up tự tải khi cần)
```

Vì sao phải là Python 3.12? Mọi buổi chốt `requires-python = ">=3.12.*"`. Từ uv 0.9, bản Python mặc định uv cài là 3.14, nên đừng dùng Python "mới nhất" cho khoá này.

Cập nhật uv sau này: `uv self update`. Cache của uv nằm ở `~/.cache/uv` (xoá được bằng `uv cache clean` khi cần giải phóng đĩa).

Nguồn: docs.astral.sh/uv/getting-started/installation, docs.astral.sh/uv/concepts/cache.

4. Cách một buổi học vận hành

Mỗi buổi là một thư mục **tự chứa**, ví dụ `buoi-07/`:

```
buoi-07/
├── BIEU-DO-...-buoi-07.pdf    tài liệu
├── code/                     điểm xuất phát (có chỗ cố tình sai – bài học là tìm ra nó)
└── lab/
```

└─ Makefile	make up check notebook down
└─ 00-nen/	môi trường chốt phiên bản + danh sách dữ liệu (đừng sửa)
└─ cham/	bộ chấm tự động

```
cd buoi-07/lab
make up          # dựng môi trường + tải dữ liệu, kiểm sha256 (lần đầu vài phút)
make notebook    # mở JupyterLab
make check       # chạy bộ chấm: đầu buổi ĐỎ, cuối buổi phải XANH
make down        # xoá môi trường + dữ liệu của buổi khi học xong
```

`make up` làm hai việc:

1. `uv sync --frozen` — cài **đúng** phiên bản thư viện ghi trong `uv.lock` vào `00-nen/.venv`. Hai học viên trên hai máy có cùng môi trường.
2. Tải dữ liệu theo `00-nen/du-lieu.toml`, **kiểm sha256** từng tệp, đặt vào `lab/du-lieu/raw/` (chỉ đọc, kèm `NGUON.txt` ghi nguồn và giấy phép).

Dữ liệu tải về được giữ ở `~/.cache/khoa-forecasting/` — `make down` rồi `make up` lại không phải tải lại; hai buổi dùng chung một tệp chỉ tải một lần. Muốn đổi chỗ cache (ví dụ sang ổ lớn hơn):

```
export KHOA_FORECASTING_CACHE=/duong/dan/o-lon/khoa-forecasting
```

Notebook

Code của khoá lưu dạng tệp `.py` có dấu `# %%` tách ô (định dạng *percent* của jupyter) — chạy được như script, mở được như notebook, và diff được trong git. `make notebook` tạo `.ipynb` từ `.py` nếu chưa có rồi mở JupyterLab. Tệp `.py` cũng mở thẳng được dưới dạng notebook bằng menu *Open as Notebook* của jupyter (đã kèm sẵn trong môi trường của buổi).

Dùng VS Code thay JupyterLab: chọn trình thông dịch `buoi-NN/lab/00-nen/.venv/bin/python`.

Khi `make up` báo lỗi

Thông báo	Làm gì
Chưa có uv	quay lại mục 3
sha256 KHÔNG KHỚP	nguồn dữ liệu đã đổi nội dung. Đừng sửa sha256 . Báo giảng viên
thiếu biến môi trường <code>...API_KEY</code>	làm mục 5
Kaggle từ chối (403)	mở link luật cuộc thi trong thông báo, bấm chấp nhận, chạy lại
HTTP 429	nguồn giới hạn tần suất — chờ vài phút rồi chạy lại
mạng chậm, tải dờ	chạy lại <code>make up</code> : tệp đã tải xong được giữ trong cache

5. API key dữ liệu (miễn phí)

Chỉ cần từ buổi có ghi trong bảng đầu trang. Key **không bao giờ** được commit vào git.

Cách lưu key — chọn một:

- Xuất biến trong `~/.bashrc` (hoặc `~/.zshrc` trên macOS):

```
export OPENAQ_API_KEY="..."
export EIA_API_KEY="..."
```

- Hoặc tạo tệp `.env` trong thư mục `lab/` của buổi (bộ tải dữ liệu tự đọc; biến đã export được ưu tiên hơn `.env`):

```
OPENAQ_API_KEY=...
EIA_API_KEY=...
```

`.env` đã có trong `.gitignore` của khoá.

OpenAQ (chất lượng không khí)

1. Đăng ký tại <https://explore.openaq.org/register>.
2. Lấy key ở <https://explore.openaq.org/account>.
3. Key gửi qua header `X-API-Key` — bộ tải dữ liệu tự làm việc này.

Giới hạn: 60 lượt/phút, 2.000 lượt/giờ. Điều khoản OpenAQ yêu cầu ghi nguồn OpenAQ và tuân theo giấy phép của từng nhà cung cấp dữ liệu. Nguồn: docs.openaq.org.

EIA (tải điện Mỹ)

1. Đăng ký tại <https://www.eia.gov/opendata/register.php> — key gửi về email.
2. Thử nhanh không cần key: EIA có key công khai `DEMO_KEY` (giới hạn rất thấp, chỉ để thử).

Nguồn: eia.gov/opendata/documentation.php.

Kaggle (buổi 23, đề A)

1. Tạo tài khoản, vào <https://www.kaggle.com/settings/api>, tạo token.
2. Đăng nhập **một** trong các cách:

```
uvx kaggle auth login          # đăng nhập qua trình duyệt
export KAGGLE_API_TOKEN="..." # hoặc biến môi trường
# hoặc lưu token vào ~/.kaggle/access_token rồi: chmod 600 ~/.kaggle/access_token
```

Tệp `kaggle.json` kiểu cũ ("Legacy API Credentials") vẫn dùng được.

3. **Mở trang luật cuộc thi và bấm chấp nhận** — ví dụ M5: <https://www.kaggle.com/competitions/m5-forecasting-accuracy/rules>. Chưa chấp nhận thì tải về bị từ chối (HTTP 403).

Luật M5 chỉ cho dùng phi thương mại và **cấm phân phối lại** dữ liệu, nên khoá không kèm dữ liệu M5 — mỗi học viên tự tải bằng tài khoản của mình. Buổi nào dùng M5 cũng có bộ dữ liệu mở dự phòng.

Nguồn: github.com/Kaggle/kaggle-cli — docs.

Hugging Face

Không cần tài khoản để tải dữ liệu và model công khai. Lớp đông người dùng chung một mạng có thể chạm giới hạn tải ẩn danh (3.000 lượt/5 phút/IP) — khi đó tạo token đọc tại <https://huggingface.co/settings/tokens> và `export HF_TOKEN="hf_..."`.

6. LLM chạy trên máy (buổi 36-37)

Buổi 36-37 chạy được **không cần trả tiền API**: dùng model mở chạy local, hoặc bộ chấm dùng bản ghi phản hồi có sẵn. Chọn một trong hai công cụ.

Ollama

```
curl -fsSL https://ollama.com/install.sh | sh # Linux và WSL (chạy lại lệnh này để cập nhật)
```

macOS: tải ứng dụng từ <https://ollama.com/download> (cần macOS 14 Sonoma trở lên).

```
ollama pull <tên-model> # tên model cụ thể ghi trong tài liệu buổi 36
ollama run <tên-model>
```

Ollama mở endpoint tương thích OpenAI tại <http://localhost:11434/v1/> (api key điền chuỗi bất kỳ, ví dụ `ollama`). Nguồn: docs.ollama.com, [OpenAI compatibility](#).

llama.cpp

```
brew install llama.cpp # macOS, Linux có Homebrew
# hoặc tải bản dựng sẵn: https://github.com/ggml-org/llama.cpp/releases
llama-server -m model.gguf -c 4096
```

`llama-server` nghe ở <http://127.0.0.1:8080>, có endpoint `/v1/chat/completions`. Nguồn: [llama.cpp install](#), [server README](#).

7. Docker (buổi 41-43)

Ubuntu / WSL không dùng Docker Desktop: theo docs.docker.com/engine/install/ubuntu (hỗ trợ Ubuntu 22.04, 24.04, 26.04). Cách nhanh cho máy học tập:

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER # đăng xuất rồi đăng nhập lại
docker compose version
```

Lưu ý bảo mật từ tài liệu Docker: nhóm `docker` "grants root-level privileges to the user".

macOS / Windows: cài Docker Desktop. Trên Windows bật *Settings* → *General* → *Use WSL 2 based engine* và *Resources* → *WSL Integration* cho bản Ubuntu (cần WSL ≥ 2.1.5).

8. Kiểm tra cuối

```
uv --version && make --version | head -1 && git --version && curl --version | head -1
```

Cả bốn lệnh in ra phiên bản là sẵn sàng cho buổi 1.

Phụ lục cho giảng viên / tác giả: công cụ của repo

Chỉ cần khi soạn khoá (sinh PDF, sinh nền buổi, đóng gói), học viên không cần.

```
# thư viện hệ thống cho weasyprint (Ubuntu) + font dùng trong PDF
sudo apt install -y libpango-1.0-0 libharfbuzz0b libpangoft2-1.0-0 libharfbuzz-subset0 \
    fonts-dejavu-core fonts-liberation
# macOS: brew install pango

uv venv .venv && uv pip install --python .venv/bin/python -r tools/requirements.txt
```

Nguồn: [weasyprint — first steps](#). Cách dùng từng công cụ: [CLAUDE.md](#), mục "Công cụ".